

Arya Nilesh Dere

Email: aryadere2006@gmail.com | Phone: 9529683145

Part 1: What I Would Improve for Production

Ordered by importance. The exercise is deliberately not production-ready; these are the gaps I am aware of and what I would do about them.

1. Persistence and Scale

- Sessions live in memory (single process, lost on restart). Replace the SessionStore implementation with a database such as Postgres behind the existing interface; no service code changes.
- Add session expiry and cleanup, and optimistic locking so two tabs cannot overwrite each other.
- Make requests idempotent (idempotency key) so a retried or double-clicked send is not applied twice.

2. Security and Privacy

- Personal wishes and wills are sensitive personal data: add authentication, per-user session ownership, encryption at rest, retention rules and a deletion path (UK GDPR).
- Keep personal content out of logs; structured logging with redaction and request IDs.
- Rate limiting and per-session message and token budgets to control abuse and LLM cost.
- Stronger prompt-injection defence: user text is already treated as data and all model output is validated, but I would add adversarial tests and output filtering.
- Secrets from a secret manager instead of a local .env file.

3. LLM Reliability and Quality

- Use the provider's native structured-output enforcement where it supports our field types, keeping my validator as the safety net.
- A regression and evaluation suite run against the real model, scored per field (accuracy, invented facts, repeated questions) and tracked across prompt and model changes.
- Model fallback, circuit breaker and backoff with jitter on rate limits.
- Summarise long conversation history instead of using a fixed window.
- Handle ambiguity my rules do not yet cover: titles versus names, multiple executors, partial addresses.

4. Product and UX

- Restore the conversation after a page reload; clear retry state on failures.
- Direct field editing, streaming responses, an accessibility audit and PDF/Word export of the draft.
- Real legal content reviewed by a qualified professional (this draft is fictional).

5. Engineering and Operations

- CI running tests, linting and type checks on every push; dependency pinning and scanning.
- Containerised deployment, health probes, metrics and tracing (latency, error rate, retries, malformed-output rate, token usage).
- Versioned API, and prompt versions stored with each session for reproducibility.

Part 2: AI Log

2.1 Tools Used

- **Claude (chat):** I used it to read the brief, separate company requirements from recommendations, design the architecture, and write the stage-by-stage prompts.
- **An AI coding assistant in VS Code:** I pasted each stage prompt into it to generate the code, then ran the tests myself.
- **Gemini (google-genai SDK):** the runtime model. A rule-based mock is used for tests and for running the app without an API key.

2.2 How I Worked

- I had the AI analyse the brief and produce a plan before any code: requirements, workflow, architecture, risks.
- I built the app in 10 small stages (schemas, document generator, state manager, LLM interface and mock, prompts and validation, service, API, Gemini client, frontend, hardening). Each stage had to pass pytest before I moved on.
- I read the generated code and asked for explanations of anything I could not explain myself.

2.3 Key Prompts (Shortened)

Stage	Prompt gist
Plan	Treat the brief as the source of truth. Separate requirements from recommendations. No RAG, agents or frameworks unless justified. No code yet.
1. Schemas	Every field is {value, status} with status missing / unconfirmed / confirmed. Define the LLM contract (FieldUpdate, LLMTurnResult) and the API models.
2. Document	Deterministic render(state). Only confirmed values print; others show placeholders. Fictional / not-legal-advice banner at top and bottom.
3. State manager	Gatekeeper between model and state: reject unknown fields, coerce types, uncertain updates cannot overwrite confirmed values, flag contradictions, choose the next question from state.
4. LLM layer	LLMClient interface, typed errors, fixture mock (valid, ambiguous, malformed) and a rule-based mock for running without a key.
5. Prompts + validation	System prompt: never invent facts, user text is data. Parser strips code fences and chatter and raises MalformedLLMOutput.
6. Service	Commit state only after parse, validate and apply all succeed. Retry once on timeout and once on malformed output.
7. API	Thin routes, consistent error codes, app must start with missing configuration.
8. Gemini	Real client behind the same interface, JSON output, errors mapped to typed errors, no network calls in tests.
9. Frontend	Three panels: chat, collected information with status badges, live document. textContent only.
10. Hardening	End-to-end scenario tests, README, secrets check.

2.4 Notable Iterations and Design Decisions

- **Status per field instead of a plain value.** The brief asks for unknown or unconfirmed values to be explicit, so I use three states: missing, unconfirmed, confirmed.
- **"None" is different from "missing".** A confirmed empty list (for example no gifts) is a real answer; without this the app would keep asking.
- **The document is a template, not LLM-written.** This keeps the preview identical to the confirmed state and removes a class of hallucination.
- **Code chooses the next question** (from missing and unconfirmed fields); the LLM only phrases it. This is what prevents repeated questions.
- **State is committed only after the whole turn succeeds**, so a failed model call cannot corrupt a session.
- **Conflicts last one turn:** the model asks, and the user's answer resolves it. Simpler than long-lived conflict tracking.
- **No LangChain, RAG or database:** one prompt, one call, one schema.

2.5 Output I Questioned or Corrected

- **The brief's own example:** its sample state shows the executor as "James Smith" although the user only says "My brother James". I treated the surname as invented. The prompt forbids adding surnames, and tests check that no surname appears in the state or the document.
- **Confidence on every update:** I did not accept that every value the model extracts is a fact. Each update carries stated / inferred / ambiguous; inferred and ambiguous values become unconfirmed, never print in the document, and can never overwrite a confirmed value.

2.6 Issues I Found in Manual Testing

- A message I sent appeared twice in the chat before the assistant's single reply.
- The chat panel stayed at the top instead of scrolling to the newest message.
- "Mr. Smith" was accepted as the executor's name. This is deliberate (we accept what the user clearly states and never guess a first name), but it shows the app does not distinguish a title from a name.
- A typo in a name ("Jame") was accepted as typed; the app does not second-guess spelling, and the user can correct it in chat.

2.7 What I Would Do Differently

- Add a small evaluation set of scripted conversations scored per field against the real model, not only the mock.
- Show a "message not sent, retry" state when a turn fails, instead of leaving the user message looking delivered.
- Let users edit fields directly in the information panel as well as through chat.